



National University
of computer and emerging sciences

Software Engineering

Rapid Remedy

Hadi Armaghan(23K-0041)

Aliyan Munawwar(23I-0641)

Sidhart Sami(23I-2527)

Abdul Rafy(23P-0560)

Systematic Benchmarking of Vector Search Architectures for Medical Decision Support

1. Executive Summary

The goal of **Clinical-RAG** is to develop a Retrieval-Augmented Generation (RAG) system that suggests medical procedures and pharmacological interventions based on patient symptoms. Unlike general-purpose RAG, medical systems require extreme precision and low latency. This project focuses on the **Backend Performance Optimization** layer, specifically comparing indexing algorithms and data chunking strategies to determine the most efficient architecture for high-stakes medical retrieval.

2. Problem Statement

The primary bottleneck in deploying RAG for healthcare is the "Retrieval Gap." Standard vector databases often use Approximate Nearest Neighbor (ANN) searches to save time, which can lead to "False Negatives" (missing a life-saving procedure in the database). Conversely, exact searches are too slow for real-time clinical use. There is currently a lack of empirical data on how **Chunk Size** (context window) and **Indexing Type** (HNSW vs. Flat) interact to affect the accuracy of medical recommendations.

3. System Architecture

The project will follow a decoupled microservices architecture:

- **Data Ingestion Layer:** Parsing unstructured medical PDFs (e.g., PubMed, Merck Manuals) into structured vector embeddings.
- **Vector Orchestration Layer:** Utilizing **PostgreSQL with pgvector** or **Milvus** to manage high-dimensional medical data.
- **Optimization Engine:** A benchmarking suite that toggles between HNSW (Hierarchical Navigable Small World) and Flat indexes to measure performance metrics.
- **Inference Layer:** A FastAPI wrapper that sends the retrieved "Top-K" medical contexts to an LLM (Gemma or Llama 3) for the final procedure suggestion.

4. Technical Specifications & Engineering Goals

4.1. Indexing Impact Analysis (The "Core" Engineering Task)

We will implement and compare two primary indexing strategies:

1. **IVF-Flat (Inverted File Index):** To test partitioned searching and its impact on medical recall.

2. **HNSW:** To test graph-based navigation. We will specifically analyze the **Construction Time vs. Query Latency** trade-off, which is critical when medical databases are updated with new research daily.

4.2. Chunking Strategy Benchmarking

Medical text is dense. We will evaluate three chunking strategies:

- **Fixed-Size Overlay:** 512-token chunks with 10% overlap.
- **Semantic Chunking:** Breaking text based on medical headings (e.g., "Contraindications," "Dosage").
- **Recursive Character Splitting:** To maintain the structural integrity of complex medical tables.

5. Implementation Roadmap (SDLC)

Phase	Focus	Key Deliverables
Research & Setup	Data Sourcing	Cleaned dataset from Kaggle/PubMed (5k+ documents).
Backend Dev	Vector Database	Implementation of pgvector/Milvus with dual-index support.
Optimization	Benchmarking	Automated scripts to measure Latency (ms) and Recall (%).
Frontend/API	User Interface	A "Doctor's Dashboard" to input symptoms and view suggestions.
Evaluation	Final Analysis	A comprehensive performance report comparing HNSW vs. Flat.

6. Evaluation Metrics (KPIs)

- **Query Latency (P99):** Goal of <150ms for retrieval across 100,000 vectors.
- **Recall @ 10:** Percentage of "true" medical procedures found using ANN vs. Exact Search.
- **Memory Efficiency:** Megabytes of RAM consumed per 1,000 indexed medical abstracts.



7. Resource Requirements

- **Compute:** Python 3.10+, Docker for database containerization.
- **Storage:** 5GB+ for vector storage and metadata.
- **APIs:** OpenAI API or a local Ollama instance for the "Generation" part.